

4 OPTIMIZATION FOR TRAINING DEEP MODELS

Optimization vs Generalization

Empirical Risk Minimization

Minimize loss on training set.

Surrogate Loss Functions

Use differentiable loss (e.g., cross-entropy) even when we care about non-differentiable loss (e.g., classification error).

Batch vs Minibatch vs Online

Type	Uses	Update frequency
Batch	Full dataset	Once per epoch
Minibatch	Subset	Multiple times per epoch
Online	Single example	Every example

Challenges in Neural Network Optimization

Ill-Conditioning

- Hessian matrix has very different eigenvalues
- Gradient descent bounces in high-curvature directions
- Need very small learning rates

Local Minima

- For large networks, most local minima have low cost
- Non-identifiability (weight space symmetry) creates many equivalent minima
- Not the main problem in practice

Saddle Points

- Points where gradient is zero but not a minimum
- Have both positive and negative eigenvalues
- Far more common than local minima in high dimensions
- Gradient descent is repelled from saddle points

Cliffs and Exploding Gradients

- Sharp regions where gradient becomes huge
- Common in RNNs (recurrent weight multiplication)
- Solution: Gradient clipping

Vanishing Gradients

- Gradients become exponentially small with depth/time
- Common in deep networks and RNNs
- Solutions: ReLU, LSTM, skip connections, careful initialization

Long-Term Dependencies (RNNs)

- To store information, network must be contractive
- Contractive dynamics $\rightarrow$ vanishing gradients
- Fundamental trade-off

Gradient Clipping

Purpose

Prevent exploding gradients.

Norm Clipping

$$\text{if } \|\mathbf{g}\| > v, \quad \mathbf{g} \leftarrow \frac{v}{\|\mathbf{g}\|} \mathbf{g}$$

Element-wise Clipping

Cap each gradient element to a maximum value.

Basic Optimization Algorithms

Gradient Descent (Batch)

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

Stochastic Gradient Descent (SGD)

$$\theta \leftarrow \theta - \eta \hat{\nabla}_{\theta} J(\theta) \quad (\text{from minibatch})$$

Learning rate schedule: Must decay to zero for convergence.

Momentum

Standard Momentum

$$\begin{aligned}\mathbf{v} &\leftarrow \alpha \mathbf{v} - \eta \nabla_{\theta} J(\theta) \\ \theta &\leftarrow \theta + \mathbf{v}\end{aligned}$$

- α is momentum coefficient (typically 0.9)
- Accelerates in directions of consistent gradient
- Helps overcome small gradients

Nesterov Momentum

$$\begin{aligned}\theta_{\text{lookahead}} &= \theta + \alpha \mathbf{v} \\ \mathbf{v} &\leftarrow \alpha \mathbf{v} - \eta \nabla_{\theta} J(\theta_{\text{lookahead}}) \\ \theta &\leftarrow \theta + \mathbf{v}\end{aligned}$$

- Evaluates gradient after applying momentum
- Better convergence rate for convex problems

Adaptive Learning Rate Methods

AdaGrad

$$\begin{aligned}\mathbf{r} &\leftarrow \mathbf{r} + \mathbf{g}^2 \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{\mathbf{r} + \epsilon}} \mathbf{g}\end{aligned}$$

- Adapts learning rates per parameter
- Larger updates for infrequent parameters
- Accumulates squared gradients (never forgets)

RMSprop

$$\begin{aligned}\mathbf{r} &\leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g}^2 \\ \theta &\leftarrow \theta - \frac{\eta}{\sqrt{\mathbf{r} + \epsilon}} \mathbf{g}\end{aligned}$$

- Uses moving average instead of accumulation
- Addresses AdaGrad's diminishing learning rates

Adam (Adaptive Moment Estimation)

$$\begin{aligned}\mathbf{s} &\leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g} && \text{(first moment)} \\ \mathbf{r} &\leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g}^2 && \text{(second moment)} \\ \hat{\mathbf{s}} &= \frac{\mathbf{s}}{1 - \rho_1^t}, \quad \hat{\mathbf{r}} = \frac{\mathbf{r}}{1 - \rho_2^t} \\ \theta &\leftarrow \theta - \eta \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \epsilon}}\end{aligned}$$

- Combines momentum with RMSprop
- Bias corrections for early steps
- Most popular optimizer for deep learning

Parameter Initialization

Importance

- Breaks symmetry between units
- Affects convergence speed and final performance
- Too small $\rightarrow$ vanishing signals
- Too large $\rightarrow$ exploding signals/saturation

Common Heuristics

Xavier/Glorot Initialization:

$$W_{ij} \sim U \left(-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}} \right)$$

He Initialization (for ReLU):

$$W_{ij} \sim \mathcal{N} \left(0, \frac{2}{n_{\text{in}}} \right)$$

Bias Initialization

- Usually set to 0
- Sometimes set to small positive number (e.g., 0.1 for ReLU)
- For output layers, can set to match marginal statistics

Pretraining

Supervised Pretraining

- Train on related task with more data
- Transfer learned features to target task

Unsupervised Pretraining

- Train layer-by-layer using unsupervised learning
- Then fine-tune with supervised learning

Why Pretraining Helps

- Better initialization
- Encodes prior knowledge
- Helps optimization

Continuation Methods

Definition

Solve a series of increasingly difficult problems.

Curriculum Learning

Present training examples in a meaningful order (easy to hard).